

PLATFORM DOCUMENTATION

7ALP's

A complete walkthrough of the storefront, admin console, and B2B wholesale portal — every login, route, and feature, as currently implemented.

Updated 2026-07-22 · Frontend: React 19 + Vite + Redux Toolkit + Tailwind 4 ·
Backend: Node.js + Express 5 + MongoDB (Mongoose)

1. Architecture Overview

The platform is a two-app monorepo: a React single-page app (`frontend/`) talking to a REST API (`backend/`) mounted under `/api/v1` . There are three independent authentication systems — Customer, Admin, and B2B — each with its own JWT stored in an httpOnly cookie, so a login in one role never grants access to another.

Frontend stack

- React 19, React Router 7
- Redux Toolkit (cart, auth, B2B draft-quote state)
- Tailwind CSS 4
- Axios (shared instance, cookie credentials on)
- Google Maps JS API (Places + Geocoding)
- Razorpay Checkout.js

Backend stack

- Express 5, MongoDB via Mongoose
- JWT + httpOnly cookies per role
- Cloudflare R2 for product images
- Razorpay Node SDK
- Resend for transactional email (OTPs / password reset) **NEW**
- Helmet, rate limiting, custom sanitize middleware

2. Logins

Customer

Mobile number + OTP, no password. A single flow handles both signup and login — if the number isn't recognized, the customer is asked for a name and an account is created automatically on OTP verification.

Field	Detail
Login route	<code>/customer/login</code> → <code>/customer/verify-otp</code>
Session	JWT in httpOnly cookie <code>customerToken</code>
Dev fallback OTP	<code>123456</code> always works until a real SMS provider is wired up

Admin

Email + password. Two roles: `Admin` and `SuperAdmin` (currently identical permissions — the distinction exists in the schema for future use).

Field	Detail
Login route	<code>/admin/login</code>
Session	JWT in httpOnly cookie <code>adminToken</code>
Seeded account	<code>7alpsadmin@7alps.com</code> / <code>7alps123</code> (run <code>npm run seed</code> in <code>backend/</code> ... see note below)
Forgot password BACKEND DONE	OTP email flow — <code>POST /api/v1/auth/forgotPassword</code> → <code>verifyOTP</code> → <code>resetPasswordAfterOTP</code> , delivered via Resend. Fully built, tested end-to-end against the live database. The <code>/admin/forgot-password</code> page in the frontend is still a static form and has not been wired to call these endpoints yet.

Careful with `npm run seed`. That script reseeds the *product catalog* from scratch (it deletes existing products first) — it is not the admin-account seeder. The admin account is seeded via `node dev-data/seedAdmin.js` directly, which is safe to re-run (it checks before creating).

B2B (marketing team)

Email + password, same shape as Admin login, but a completely separate account collection. B2B accounts are **created by an admin** — there is no self-signup. Each B2B login represents an internal marketing-team member who sells in bulk on behalf of a business buyer.

Field	Detail
Login route	<code>/b2b/login</code>
Session	JWT in httpOnly cookie <code>b2bToken</code>
Account creation	Admin → B2B Team → Add member
Seeded account	<code>b2bpartner@7alps.com</code> / <code>7alps123</code> (<code>npm run seed:b2b</code>)
Forgot password NEW	Same OTP flow as Admin, built from scratch this round — <code>POST /api/v1/b2b/forgotPassword</code> → <code>verifyOTP</code> → <code>resetPasswordAfterOTP</code> , delivered via Resend. New <code>passwordResetToken</code> / <code>passwordResetExpires</code> fields added to the B2B member schema. Backend-complete and tested end-to-end; no frontend page wired to it yet.

3. Route Map

Public (no login)

Route	Page
/	Home — hero, categories, featured products, process, testimonials, FAQs
/products	Full catalog with filters
/products/:id	Product detail — gallery, info, ingredients, reviews
/our-process	Sourcing/production story
/why-7alps	Brand positioning / comparison
/global-trade	Export/trade information
/partners	Partnerships page
/b2b-portal	Wholesale marketing/landing page (static content, not the logged-in B2B portal)
/contact	Contact form + map embed
/cart	Shopping cart (guest or logged-in)

Customer (requires customer login)

Route	Page
/checkout	Shipping address (with map picker) + payment method + place order
/customer/orders	Order history, with a product photo per line item NEW
/customer/profile	Personal details + address book (with map picker)

Admin (requires admin login)

Route	Page
/admin	Dashboard — customers, orders, products, revenue
/admin/products	Product CRUD, variants, image upload
/admin/categories	Category CRUD
/admin/orders	All orders (customer + B2B), status updates
/admin/customers	Customer list with order/spend stats
/admin/reviews	Review moderation
/admin/b2b	B2B Team — members + quote approval

B2B (requires B2B login)

Route	Page
/b2b	Dashboard — quotes, orders, sales stats
/b2b/catalog	Browse products, propose bulk price/qty
/b2b/request-quote	Buyer details (with map picker) + submit + quote history
/b2b/orders	Orders created from approved quotes, searchable by order ID or product NEW

4. Feature Walkthrough

Customer-facing storefront

- **Browse & filter** — category, price, search, rating sort.
- **Cart** — guest cart in localStorage; merges into the server cart automatically on login.
- **Checkout** — shipping address form with an embedded Google Maps picker (search or drag a pin to auto-fill address fields); choice of **Cash on Delivery** or **Pay Online (Razorpay)**.
- **Orders** — history with status, itemized breakdown (now including each item's product photo), shipping address.
- **Profile** — editable name/email; a real address book (add/edit/delete/set-default), each entry fillable via the same map picker.
- **Reviews** — logged-in customers can rate & review a product they're viewing; reviews are held as **Pending** until an admin publishes them.

Admin console

- **Dashboard** — live counts: customers, orders, products, revenue.

- **Products** — full CRUD, multiple variants per product (label/price/MRP/stock), multi-image upload to Cloudflare R2.
- **Categories** — add/edit/activate/delete; delete is blocked if products still reference the category, with a clear error explaining why.
- **Orders** — every order (storefront and B2B) in one place; inline status changes (Confirmed → Processing → Shipped → Delivered/Cancelled).
- **Customers** — every registered customer with their order count and lifetime spend.
- **Reviews** — moderate: publish, flag, or delete any submitted review.
- **B2B Team** — create marketing-team logins; see each member's total bulk orders and sales; review a pending quote (adjust quantity/price if needed), approve it — which atomically decrements stock and creates a real order — or reject it with a reason. The Quotes list endpoint now also accepts `search` (member name/email/business/product) and `dateFrom` / `dateTo` query params for filtering, in addition to `status` **NEW** — the admin UI still only wires up the `status` dropdown, so exposing search/date in the tab itself is a small follow-up.

B2B wholesale portal

- **Catalog** — the same live product catalog; the member picks a variant, a bulk quantity (10+ units), and proposes their own discounted unit price (capped at the listed retail price).
- **Request Quote** — the products picked in Catalog collect into a draft; the member adds the buyer's business name and delivery address (map picker included) and submits it for admin approval. Past quotes and their status are listed below the form.
- **Orders** — once admin approves a quote, it becomes a real order here; now searchable by order ID or product name **NEW**, alongside the existing status filter.
- **Dashboard** — quote counts by status, total orders, total sales.

5. Payments

Cash on Delivery is the default and always available. **Razorpay is fully live and has been verified with a real completed test transaction** — order creation, the checkout modal, signature verification, and order confirmation all work end to end. If `RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET` in `backend/.env` are ever blanked out, attempting to pay online returns a clear "not configured yet" error rather than failing silently — it never crashes the server either way.

How it works: the backend computes the charge amount itself from the customer's live server-side cart — it never trusts a client-supplied amount, which is the standard payment-security practice (a client could otherwise tamper with what it pays). `POST /customer/payments/razorpay-order` creates the Razorpay order sized to that amount; the frontend opens Razorpay's Standard Checkout modal with it; on success, `POST /customer/payments/razorpay-verify` checks the HMAC-SHA256 signature Razorpay returns before creating the real order, decrementing stock, and clearing the cart — mirroring the same transaction the COD path uses.

6. Email (Resend) NEW

All transactional email — signup OTPs, and now both the Admin and B2B forgot-password OTP flows — sends through [Resend](#) instead of the previous Gmail/Nodemailer setup. The change is isolated to `backend/utils/email.js`, which keeps the same `sendEmail({ email, subject, message })` signature, so no caller had to change.

Field	Detail
Config	<code>RESEND_API_KEY</code> + <code>RESEND_FROM_EMAIL</code> in <code>backend/.env</code>
Sender address	<code>7Alps <7alps@lanvaus.com></code> — <code>lanvaus.com</code> is verified with Resend; a dedicated 7alps domain is expected soon and this address can move to it with a one-line <code>.env</code> change
Verified end-to-end	Live test emails sent to an arbitrary external address and to a <code>7alps.com</code> address on 2026-07-22 — both accepted and delivered by the API, confirming domain verification unlocked sending to real recipients (not just the account owner)

Domain verified — sandbox restriction lifted. Earlier testing showed the Resend account could only deliver to its own registered address until a domain was verified. With `lanvaus.com` now verified, OTP emails send to any recipient. When the dedicated 7alps domain is ready, update `RESEND_FROM_EMAIL` to use it instead.

7. Testing Reference

What	Value
Admin login	<code>7alpsadmin@7alps.com</code> / <code>7alps123</code>
B2B login	<code>b2bpartner@7alps.com</code> / <code>7alps123</code>
Customer login	Any 10-digit mobile number, OTP <code>123456</code>
Razorpay test card (recommended)	<code>5267 3181 8797 5449</code> (Mastercard) — any future expiry, any 3-digit CVV
Razorpay test OTP	Any 4-10 digit number (e.g. <code>1234</code>) — it's a test-mode simulation, not a fixed code
Password-reset OTP (Admin/B2B) NEW	Emailed via Resend from <code>7alps@lanvaus.com</code> , delivered to the real account email (see Section 6)

Avoid the generic `4111 1111 1111 1111` Visa test card — Razorpay's test environment treats it as a foreign-issued card, and most test accounts (this one included) have international card acceptance disabled by default, so it fails with `international_transaction_not_allowed`. The Mastercard number above is domestic and works reliably.

8. Known Limitations

- **Admin/B2B "Forgot Password" pages are still frontend UI shells** — the backend OTP flow is now fully built and tested for both roles (Section 2), but neither `/admin/forgot-password` nor `/b2b/forgot-password` is wired to call it yet; the forms still just render with no submit handler.
- **Admin Quotes tab filtering is backend-only** — `GET /api/v1/admin/quotes` now accepts `search` / `dateFrom` / `dateTo` , but the B2B Team page's Quotes tab UI still only exposes the pre-existing `status` dropdown.
- **SMS OTP delivery is not wired up** — OTPs are logged to the server console in development, and `123456` is always accepted as a fallback until a real SMS provider (e.g. MSG91, Twilio) is integrated.
- **The home page's decorative "Shop by category" tiles are static content** (curated imagery in a fixed 3-tile layout), separate from the real, admin-managed Category system used everywhere else (product forms, filters, B2B catalog).
- **No order/quote-status confirmation emails** — order placement, quote approval/rejection, etc. remain visible in-app only; Resend is currently used only for the OTP flows above.
- **Email sender is on a shared parent-company domain** — OTPs currently send from `7alps@lanvaus.com` . A dedicated 7alps domain is expected soon; once verified with Resend, update `RESEND_FROM_EMAIL` in `backend/.env` to move onto it.